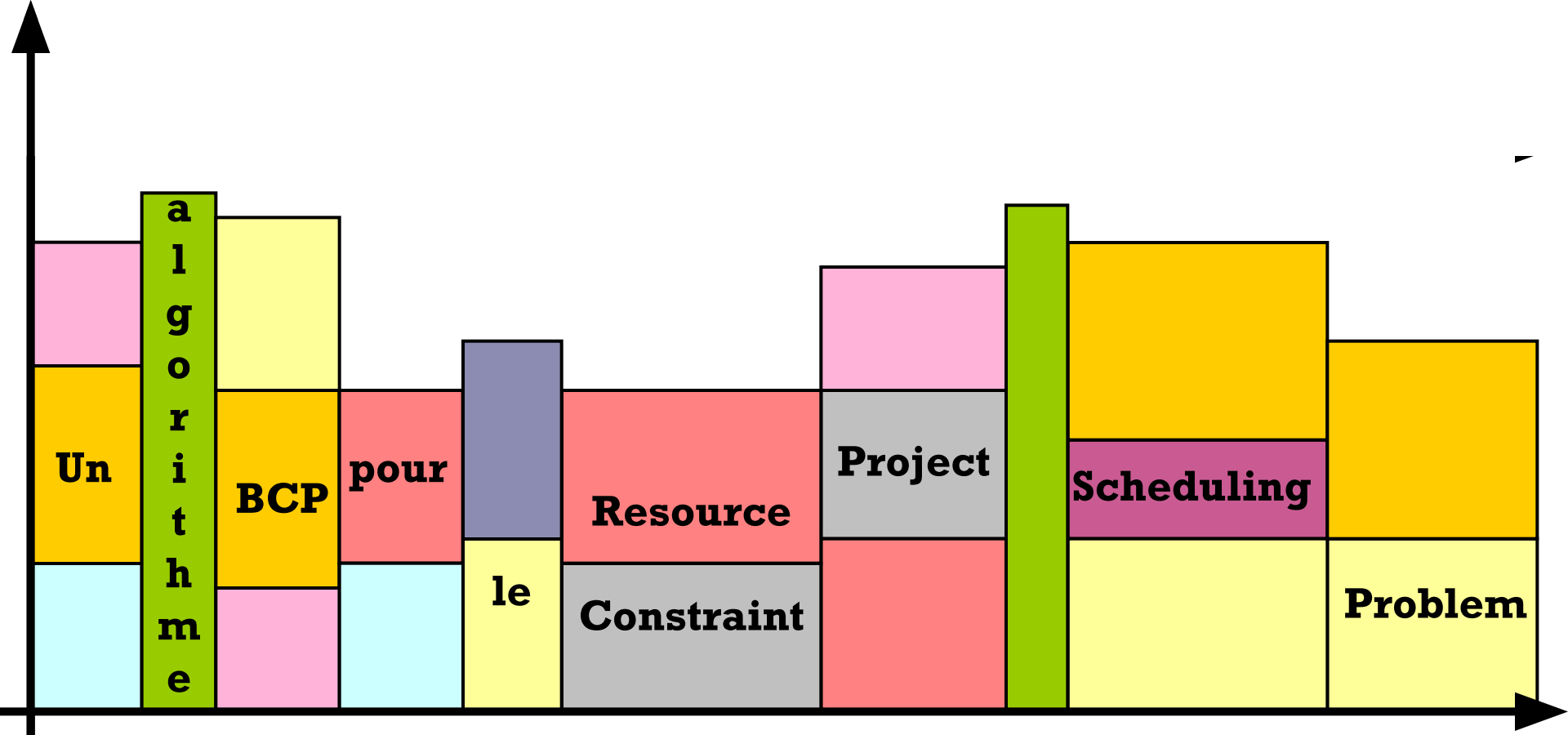




Jean Damay

Laboratoire LIMOS

Université Blaise Pascal

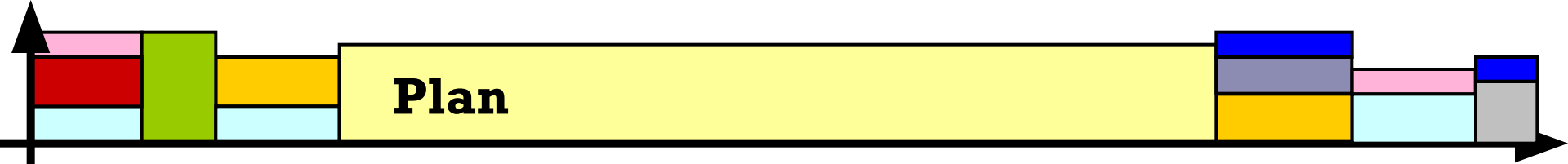
Clermont II

Pierre Fouilhoux

Laboratoire LIP6

Université Pierre et Marie Curie

Paris 6



A- Définition du RCPSP

B- Antichaînes

C- Une formulation PLNE pour le cas préemptif

D- Un algorithme BCP pour le cas préemptif

E- Formulation et pistes pour le cas non-préemptif



A- Définition du RCPSP

Un projet (Une instance du RCPSP)

- Un ensemble $V=\{1,...,n\}$ d'activités (ou tâches)
 - Une durée d'exécution $p_i, i \in V$
 - Un ordre partiel sur les activités.

Contraintes de précédence

- Un ensemble $R=\{1,...,K\}$ de ressources limitées renouvelables
 - Quantité disponible à tout moment $R_k, k \in R$
 - Quantité nécessaire r_{ik} de ressource $k \in R$ pour une activité $i \in V$

Contraintes de ressources

- Un ensemble $I \subset V$ des activités non-préemptives
(c'est-à-dire des activités devant être exécutées en un seul bloc)

Contraintes de non-préemptivité



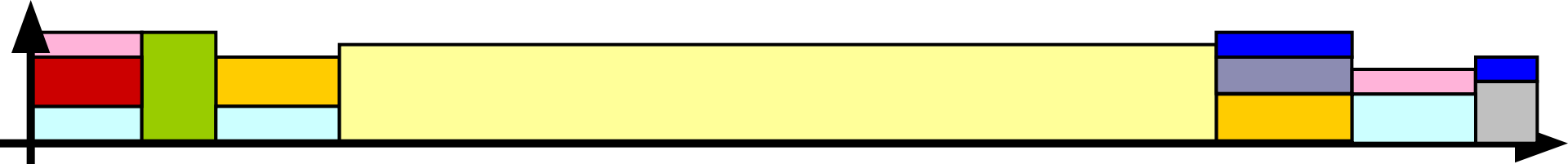
A- Définition du RCPSP

Resource Constraint Project Scheduling Problem RCPSP

Déterminer un ordonnancement des tâches ou d'un fractionnement des tâches en respectant

- les contraintes de précédences
(exécution de l'activité i finie avant celle de l'activité j)
- les contraintes de ressources
(les ressources nécessaires à des tâches exécutées simultanément ne doit pas exéder les quantités de ressources disponibles)
- les contraintes de non-préemptivité
(certaines activités ne peuvent être fractionnées)

en minimisant la durée total du projet (makespan).



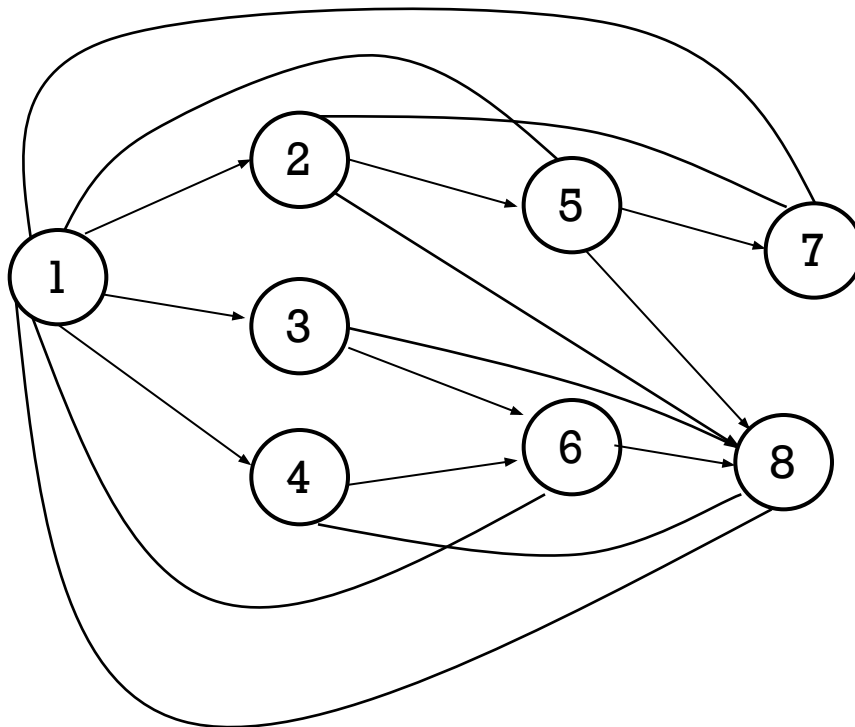
Graphe de précedence

L'ordre partiel sur les activités

→ un graphe de précedence $G=(V,E)$

tq $\iff$ l'arc $(i,j) \in E$

l'exécution de l'activité i terminée avant le début de j

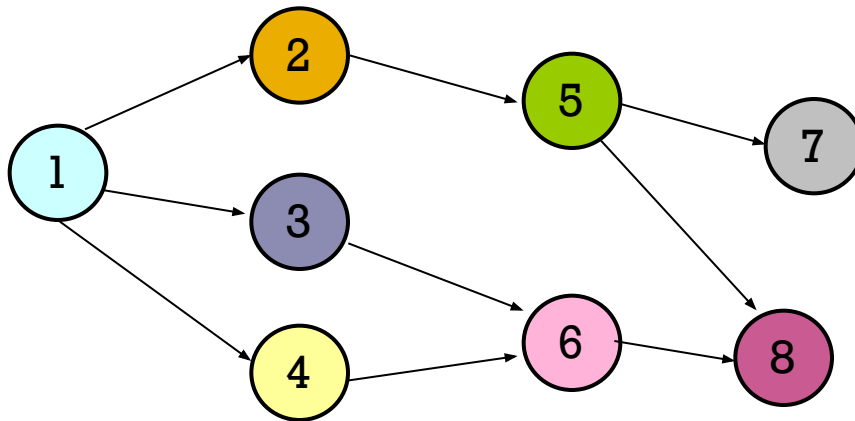


On étend cette définition
du graphe de précedence
à $\bar{G}=(V,\bar{E})$
où $\bar{E}$ est la fermeture transitive de E

A- Définition du RCPSP

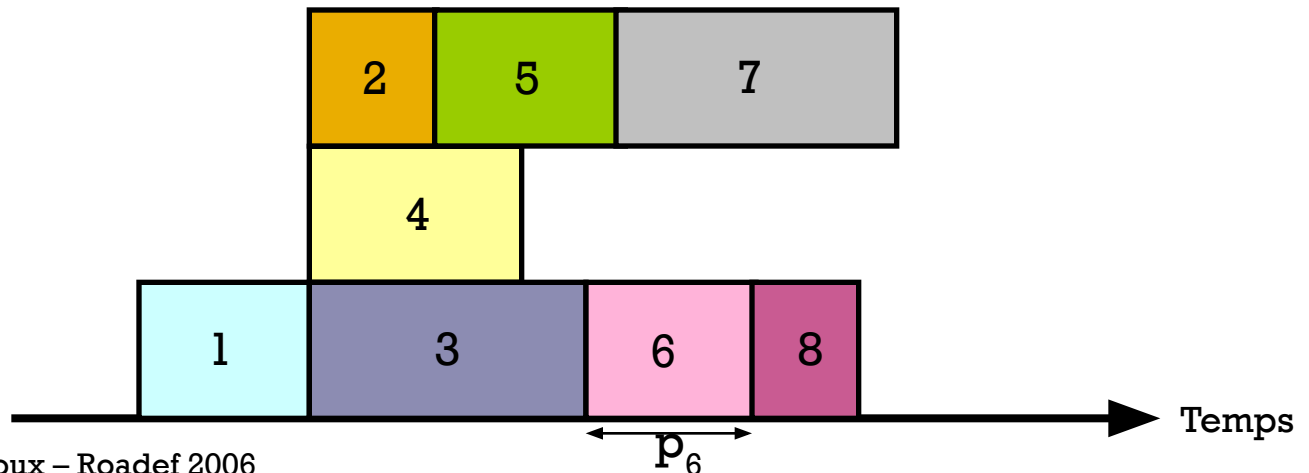
Cas: Ressources infinies et non-préemptif

$(R_k = +\infty \text{ pour tout } k \in R \text{ et } I=V)$



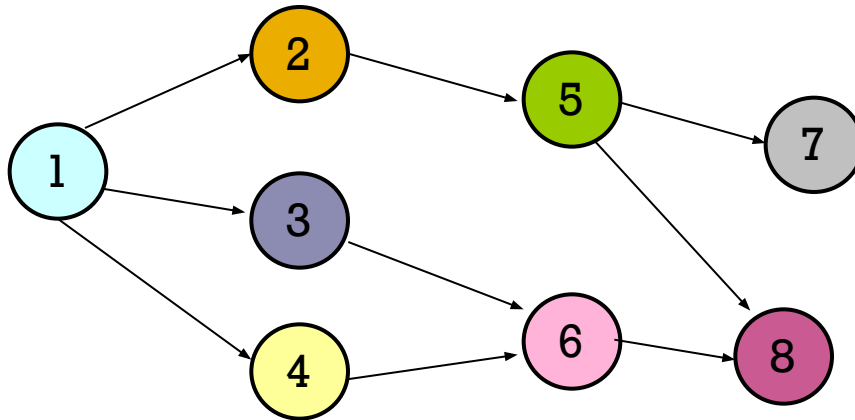
Une solution admissible peut être trouvée par un simple parcours en largeur donnant une liste topologique

$p_1=3$
 $p_2=2$
 $p_3=6$
 $p_4=5$
 $p_5=4$
 $p_6=3$
 $p_7=6$
 $p_8=2$



A- Définition du RCPSP

Ajoutons une ressource limitée

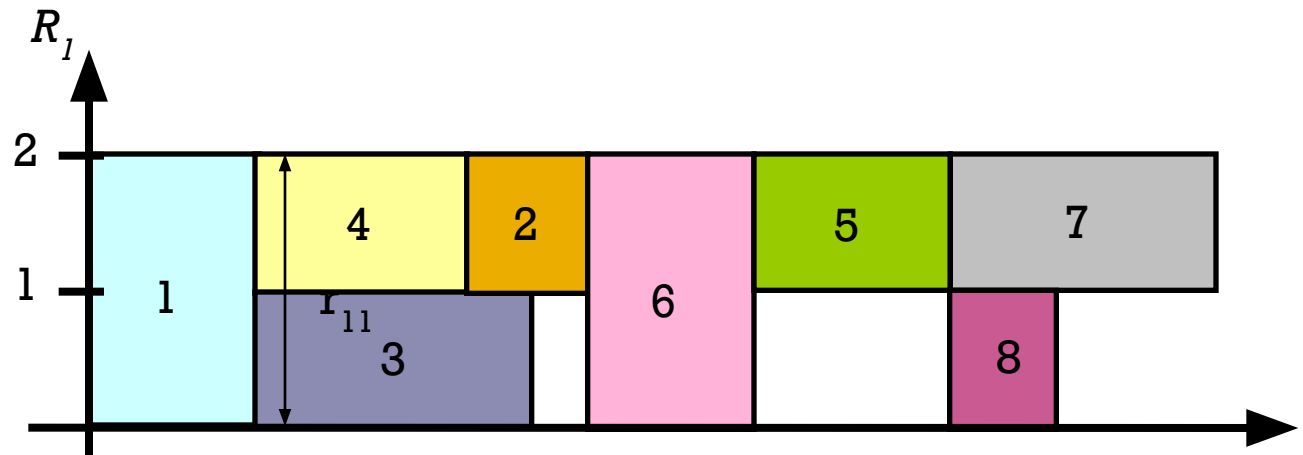


$p_1=3$	$r_{11}=2$	$R_1=2$
$p_2=2$	$r_{21}=1$	
$p_3=6$	$r_{31}=1$	
$p_4=5$	$r_{41}=1$	
$p_5=4$	$r_{51}=1$	
$p_6=3$	$r_{61}=2$	
$p_7=6$	$r_{71}=2$	
$p_8=2$	$r_{81}=1$	

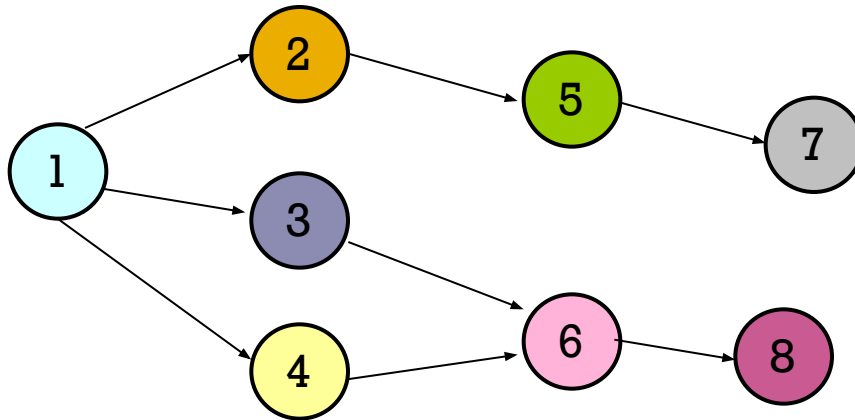
A tout moment

$$\sum_{i \in V} r_{ik} \leq R_k$$

 exécutées en même temps

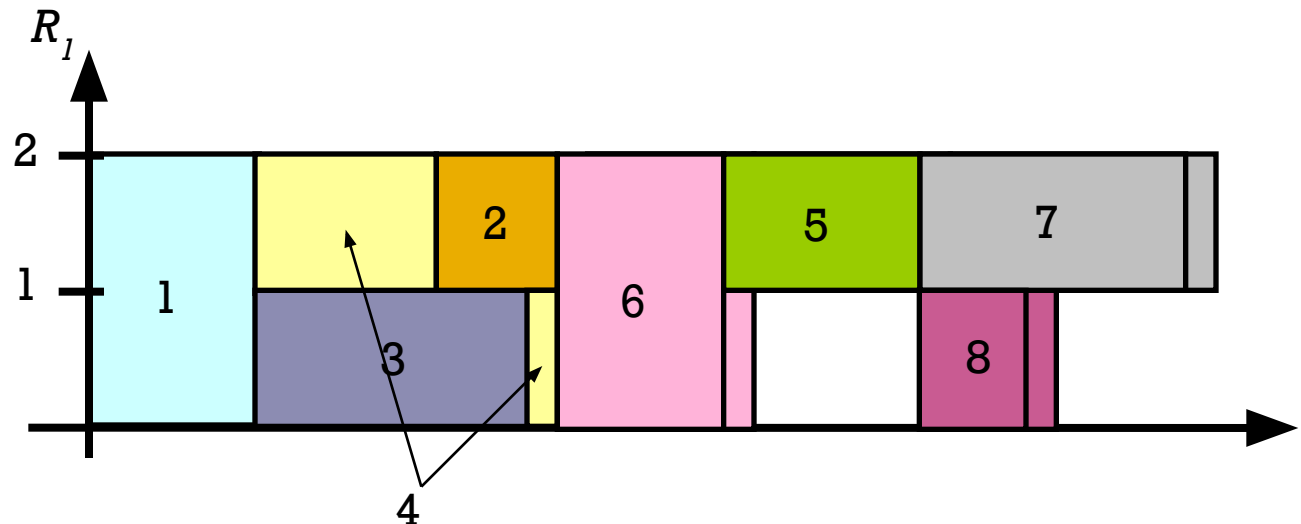


A- Définition du RCPSP



$p_1=3$	$r_{11}=2$	
$p_2=2$	$r_{21}=1$	
$p_3=6$	$r_{31}=1$	
$p_4=5$	$r_{41}=1$	
$p_5=4$	$r_{51}=1$	
$p_6=3$	$r_{61}=2$	
$p_7=6$	$r_{71}=2$	
$p_8=2$	$r_{81}=1$	
		$R_1=2$

Ajoutons la
préemptivité
de l'activité 4

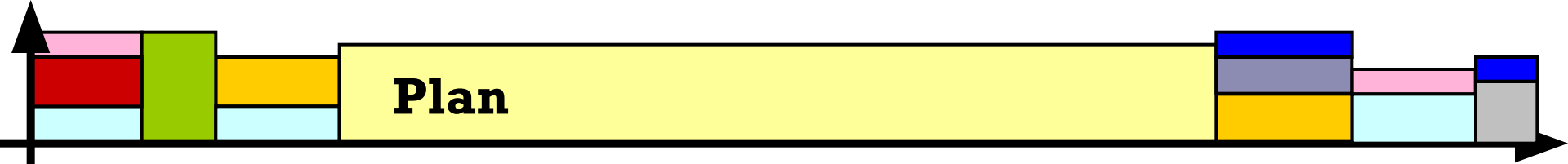




A- Définition du RCPSP

Etat de l'art

- Nombreux articles sur des méthodes approchées et sur des algorithmes exacts dans des cas polynomiaux
- Si le temps peut être discrétisés, il existe des travaux sur des résolutions exactes (par exemple par relaxation lagrangienne: Möhring, Schulz, Strok, Uetz - 2002)
- Etude polyédrale d'un cas très particulier (Coll, Ribeiro, De Souza - 2003)
- Travaux utilisant une modélisation par « antichaines »
 - Mingozzi, Maniezzo, Ricciardelli, Bianco – 95
 - Brucker, Knust – 98
 - Damay, Sanlaville - 2005



A- Définition du RCPSP

B- Antichaînes

C- Une formulation PLNE pour le cas préemptif

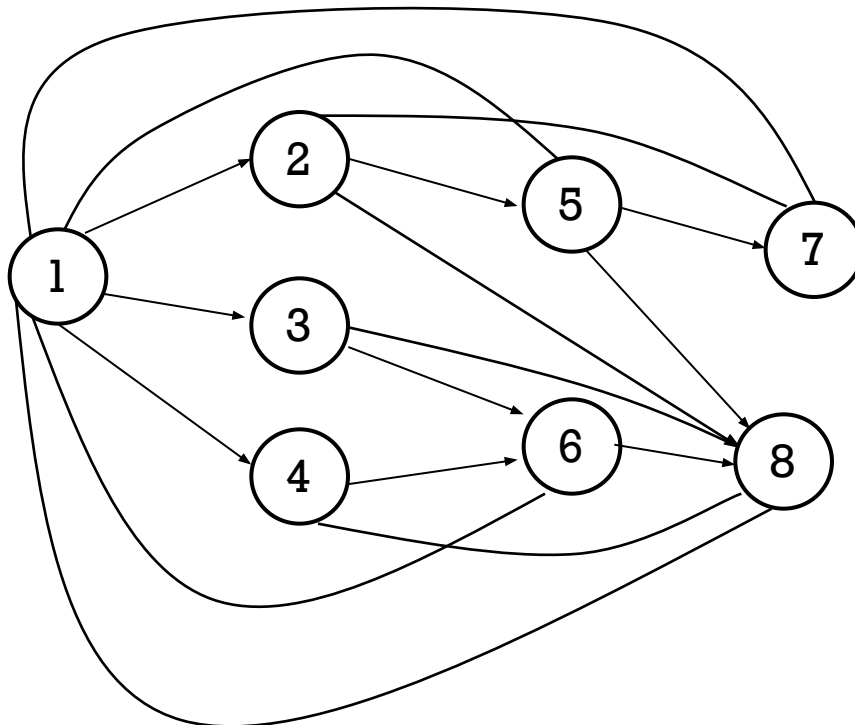
D- Un algorithme BCP pour le cas préemptif

E- Formulation et pistes pour le cas non-préemptif

B- Antichaînes

Antichaîne: ensemble de sommets sans chemin
reliant une paire de ces sommets (dans un graphe orienté).

Dans le graphe orienté de précédence $G=(V,E)$,
une antichaîne $\Rightarrow$ ensemble de tâches
sans ordre de précédence

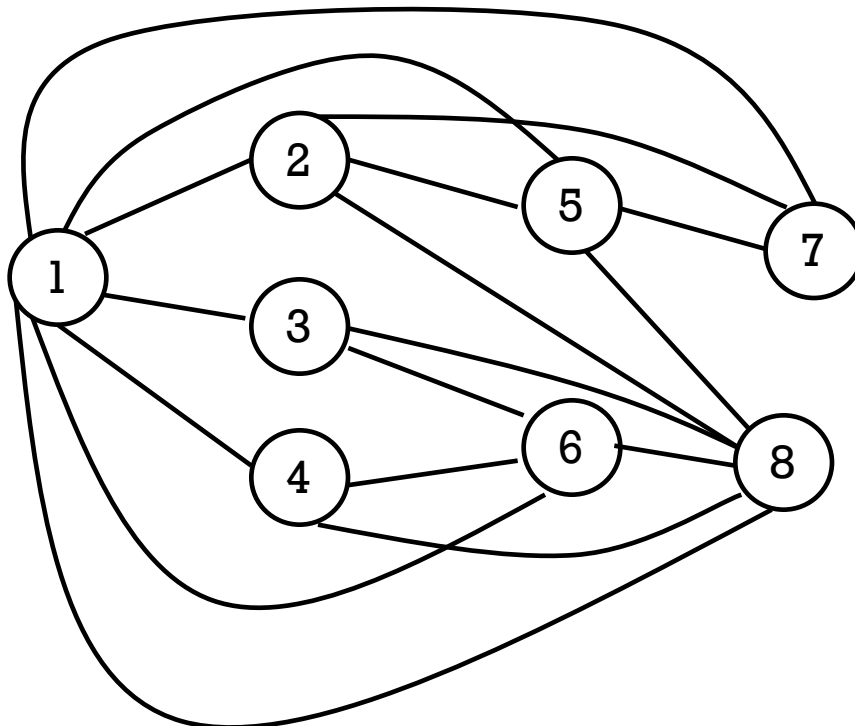


Dans la fermeture transitive
 $\bar{G}=(V,\bar{E})$ de E , il s'agit donc d'un
ensemble de sommets non reliés
par des arcs

B- Antichaînes

Antichaîne: ensemble de sommets sans chemin
reliant une paire de ces sommets (dans un graphe orienté).

Dans le graphe orienté de précédence $G=(V,E)$,
une antichaîne $\Rightarrow$ ensemble de tâches
sans ordre de précédence



Dans la fermeture transitive
 $\bar{G}=(V,\bar{E})$ de E , il s'agit donc d'un
ensemble de sommets non reliés
par des arcs

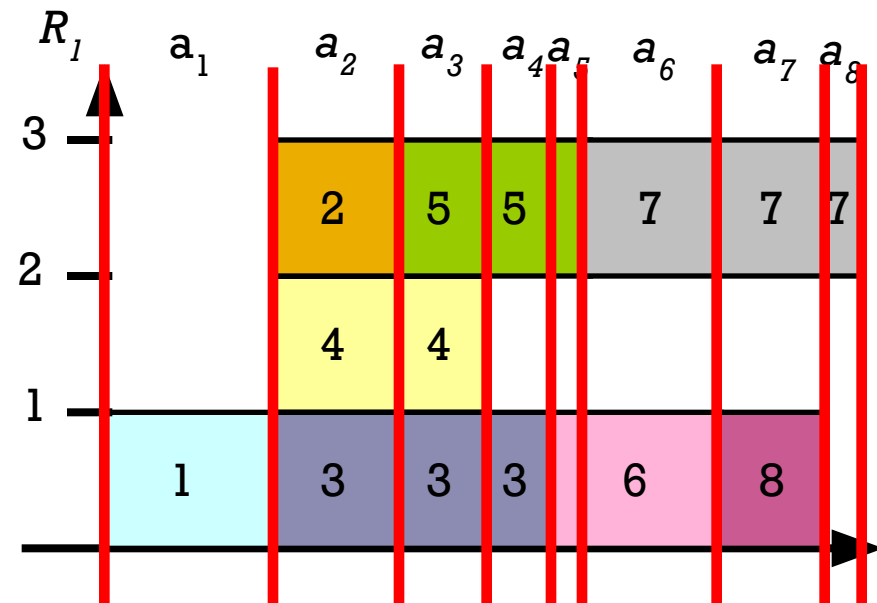
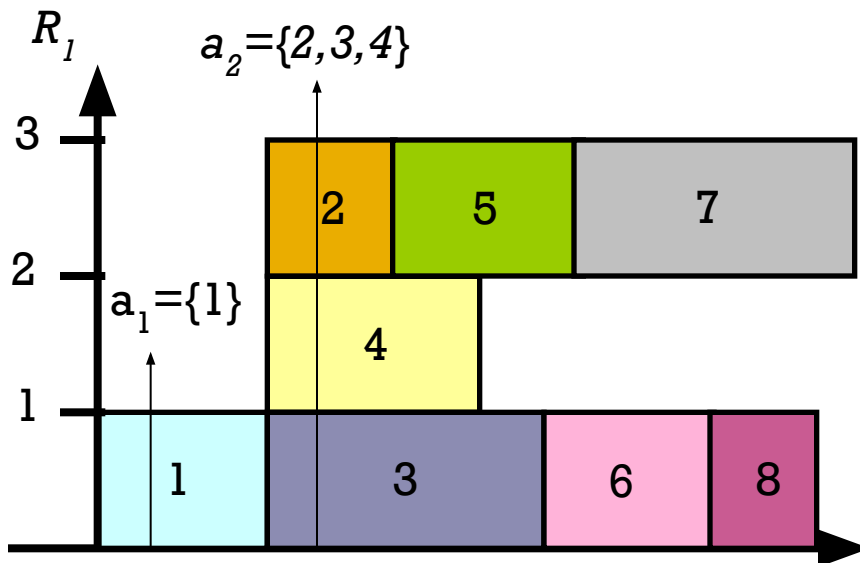
En supprimant l'orientation
de $\bar{G}=(V,\bar{E})$ de E ,
une antichaîne est alors
un stable de ce graphe

B- Antichaînes

Antichaîne valide: ensemble de tâches sans ordre de précedence et respectant les contraintes de ressources

(ie. une antichaîne a telle que $\sum_{i \in a} r_{ik} \leq R_k, \forall k \in R$).

Antichaîne valide $\Rightarrow$ ensemble de tâches pouvant être exécutées simultanément



B- Antichaînes

Relation d'ordre sur l'ensemble $\mathcal{A}$ des antichaînes valides.

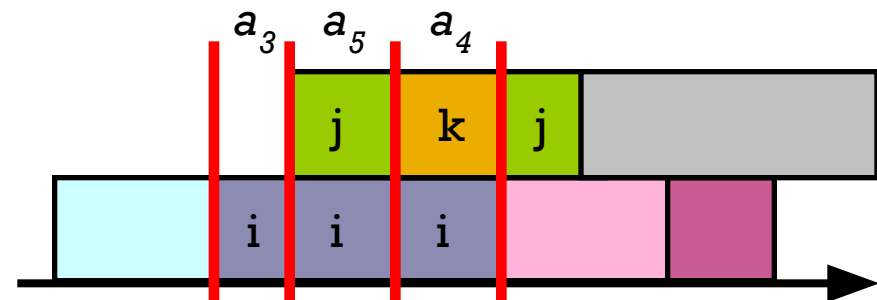
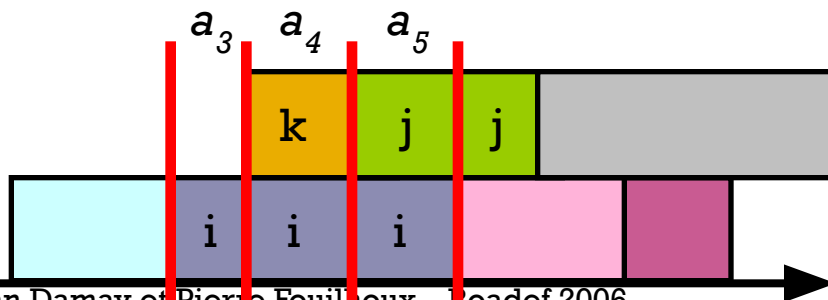
$a < a' \iff$ il existe un arc (i,j) dans $\bar{E}$
avec i tâche de a et j tâche de a'

Séquence ordonnée d'antichaînes

$(a_1, a_2, \dots, a_p)$ telle qu'il n'existe pas d'indice $l < l'$ avec $a_l < a_{l'}$

Séquence ordonnée i -consécutive d'antichaînes

$(a_1, a_2, \dots, a_p)$ séquence ordonnée d'antichaînes
où l'ensemble des antichaînes qui contiennent
une même tâche i sont consécutives

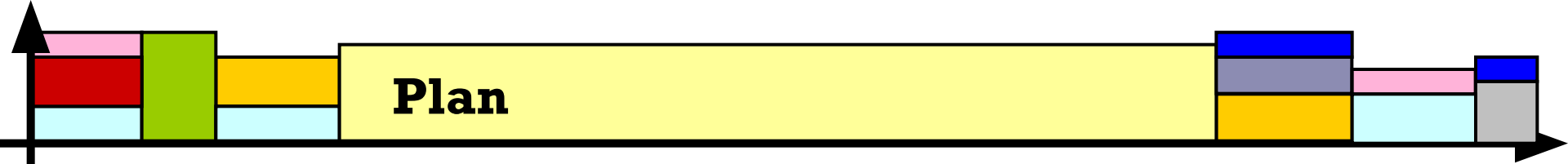


The diagram illustrates the structure of a B-chain antibody. It features a central yellow rectangle labeled "B- Antichaînes". To the left of this central region, there are three stacked colored rectangles: red (top), green (middle), and cyan (bottom). To the right of the central region, there are three stacked colored rectangles: blue (top), grey (middle), and yellow (bottom). Further to the right, there are two more stacked colored rectangles: pink (top) and cyan (bottom). The entire structure is flanked by two black arrows pointing outwards, one on the left and one on the right.


$$\sum_{a/i \in a} z_a = p_i$$

Thm:

- ordonnée (contraintes d'ordre)
 - d'antichaînes valides (contraintes de ressources)
 - i -consécutive $\forall i \in I$ (contrainte de non-préemptivité)
- $$- \sum_{a/i \in a} z_a = p_i, \forall i \in V$$
- et avec $\sum_{l=1}^p z_{a_l}$ minimum



A- Définition du RCPSP

B- Antichaînes

C- Une formulation PLNE pour le cas préemptif

D- Un algorithme BCP pour le cas préemptif

E- Formulation et pistes pour le cas non-préemptif

Graphe d'antichaines

Soit $\Gamma = (\mathcal{A}, \mathcal{C})$ un graphe orienté

- avec $\mathcal{A}$ l'ensemble des antichânes valides
- (a, a') arc de $\mathcal{C}$ si $a < a'$

- Γ contient un nombre exponentiel de sommets

Mais on travaillera sur $\Gamma(A)$ le sous-graphe de Γ
induit par A « petit » sous-ensemble d'antichânes

- Si Γ ne contient pas de circuit, une liste topologique issue de Γ donne une solution réalisable pour le RCPSP
Mais Γ contient nécessairement des circuits.

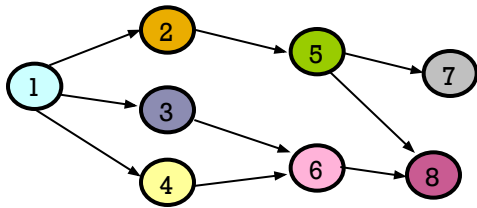
C- Formulation PLNE

Dans le cas où toutes les tâches sont préemptives

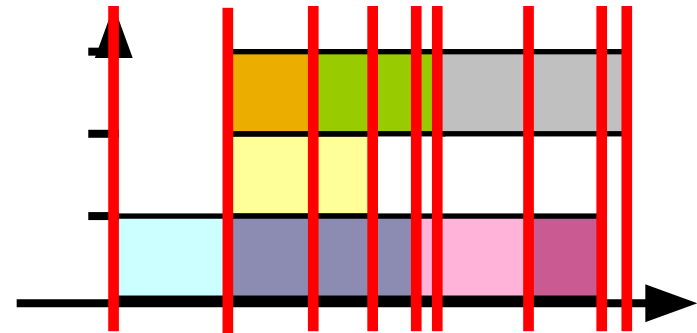
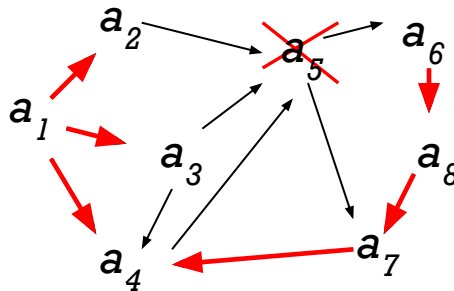
Thm: Le problème de RCPSP se ramène à trouver une séquence $s=(a_1, \dots, a_p)$ d'antichaînes valides

avec $\Gamma(s)$ acyclique

$$\sum_{a/i \in a} z_a = p_i, \forall i \in V \quad \text{et} \quad \sum_{l=1}^p z_{a_l} \text{ minimum.}$$



Ressources





C- Formulation PLNE

Variables

- A chaque antichaîne valide a , on associe

z_a variable de durée de l'antichaîne

x_a variable de décision $x_a = 1$ si l'antichaîne est dans la solution
0 sinon

- Liens entre les deux variables:

Si $x_a = 0$ alors $z_a = 0$



C- Formulation PLNE

Formulation PLNE

$$\text{Min } \sum_{a \in A} z_a$$

$$\sum_{a \in A / i \in a} z_a = p_i, \forall i \in V,$$

$$z_a \leq M \cdot x_a, \forall a \in A,$$

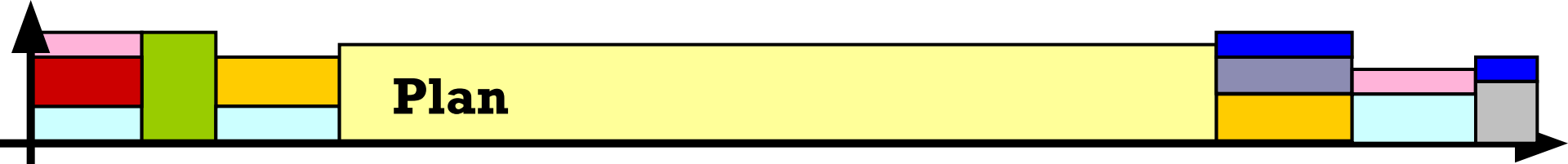
$$\sum_{a \in C} x_a \leq |C| - 1, \text{ pour tout circuit } C \text{ de } \Gamma(A),$$

$$z_a \geq 0, \forall a \in A, \text{ et } x_a \in \{0,1\}, \forall a \in A.$$

Pour obtenir la durée
des antichaînes

Contraintes de couplages

Briser les circuits



A- Définition du RCPSP

B- Antichaînes

C- Une formulation PLNE pour le cas préemptif

D- Un algorithme BCP pour le cas préemptif

E- Formulation et pistes pour le cas non-préemptif



D- Un algorithme BCP

Algorithme Branch-and-Cut-and-Price

- 1) Initialisation: on prend un ensemble d'antichaînes correspondant à une solution réalisable (les antichaînes singletons) $\rightarrow A$
- 2) Trouver une solution optimale x^* de la relaxation linéaire restreinte à A .
- 3) Pricing: Rechercher une antichaîne valide a de coût réduit $r(a) < 0$
- 4) Séparation: Rechercher s'il existe des contraintes de circuits violées à partir du graphe $\Gamma(A \boxplus \{a\})$
- 5) Si toutes les antichaînes hors de la relaxation ont un coût réduit ≥ 0 et s'il n'y a pas de contraintes de circuits violées, alors (la solution est optimale pour relaxation linéaire du PLNE)
Si les variables x sont entières alors FIN
Sinon Branchement (et/ou ajout de coupes)
Sinon $A \leftarrow A \boxplus \{a\}$ et on ajoute les contraintes de circuits à la relaxation
- 6) Aller en 2.



D- Un algorithme BCP

Pricing: Trouver une antichaîne valide de plus petit coût réduit
(car si son coût réduit est ≥ 0 , toutes les autres sont dans le même cas)

$$\text{Min } \sum_{a \in A} z_a$$

$$\sum_{a \in A / i \in a} z_a = p_i, \forall i \in V,$$

$$z_a \leq M \cdot x_a, \forall a \in A,$$

$$\sum_{a \in C} x_a \leq |C| - 1, \text{ pour tout circuit } C \text{ de } \Gamma(A),$$

$$z_a \geq 0, \forall a \in A,$$

$$0 \leq x_a \leq 1, \forall a \in A.$$

y_i

Dans la relaxation linéaire
restreinte à un ensemble A d'antichaînes
la variable d'une nouvelle antichaîne
n'apparaît que dans ces contraintes

Coût réduit d'une
nouvelle antichaîne:

$$r(a) = 1 - \sum_{i \in a} y_i$$



D- Un algorithme BCP

Pricing: Trouver une antichaîne valide de plus petit coût réduit
(car si son coût réduit est ≥ 0 , toutes les autres sont dans le même cas)

Coût réduit d'une nouvelle antichaîne: $r(a) = 1 - \sum_{i \in a} y_i$

Trouver une antichaîne valide qui maximise $\sum_{i \in a} y_i$

Trouver un stable S dans $\bar{G}=(V, \bar{E})$ de poids maximum par rapport au poids y_i
tel que les tâches de S vérifie les contraintes de ressources.

Résoudre $\left\{ \begin{array}{l} \text{Max } \sum_{i \in V} y_i x(i) \\ x(i) + x(j) \leq 1, \forall ij \in E \\ \sum_{i \in V} r_{ik} x(i) \leq R_k \\ x(i) \in \{0,1\}, \forall i \in V. \end{array} \right.$

C'est un stable avec des contraintes
de knapsak (un knapstable)

D- Un algorithme BCP

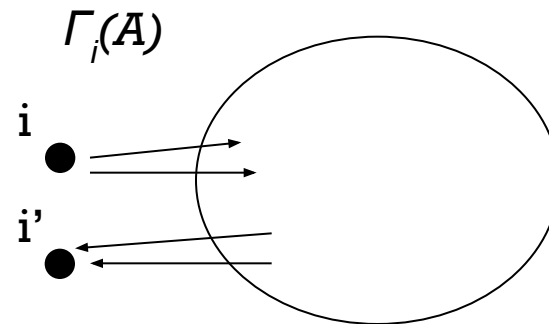
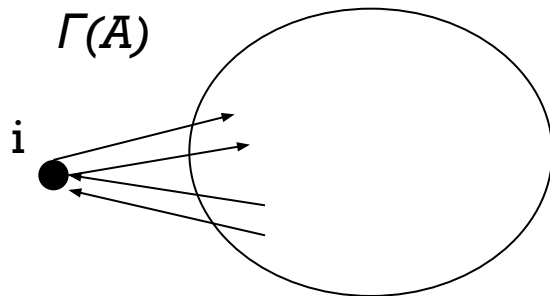
Séparation de contraintes de circuits Trouver la contrainte la plus violée

$$\sum_{a \in C} x_a \leq |C| - 1, \text{ pour tout circuit } C \text{ de } \Gamma(A),$$

$$x'(a) = 1 - x(a) \longrightarrow \sum_{a \in C} x'_a \geq 1$$

La séparation revient à trouver un plus petit circuit dans $\Gamma(A)$ selon le poids x'

Pour chaque tâche i , on recherche un plus court chemin de i à i' dans $\Gamma(A)$

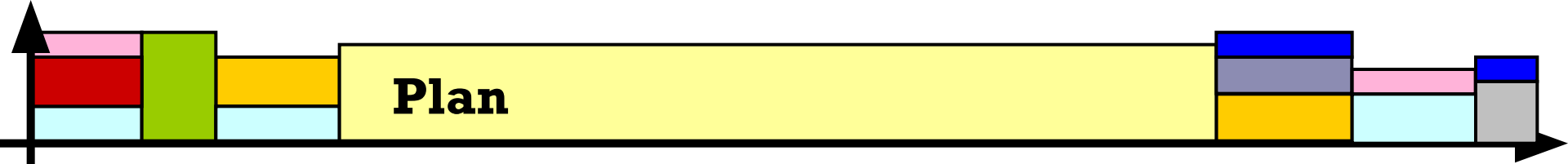




D- Un algorithme BCP

En pratique

- Code sous Abacus en utilisant Cplex comme solveur linéaire
- Le pricing résolu en fournissant le PLNE du knapstable directement à Cplex
- Manipulation d'un ensemble réduit d'antichaînes dit « variables actives » en suivant la valeur de leur coût réduit
- On travaille sur les antichaîne maximale au sens de l'inclusion (on peut le faire en changeant la fonction objectif du PLNE du pricing)
- Branchement en se basant sur la règle de Ryan et Foster:
ici: trouver une paire de tâche (i,j) et coupé l'espace des solutions entre celles dont les antichaînes contiennent toujours i et j ensemble ne contiennent jamais i et j en même temps
- Ajout de coupes: en fait, il s'agit du polytope des sous-graphes acycliques qui a déjà été étudiés



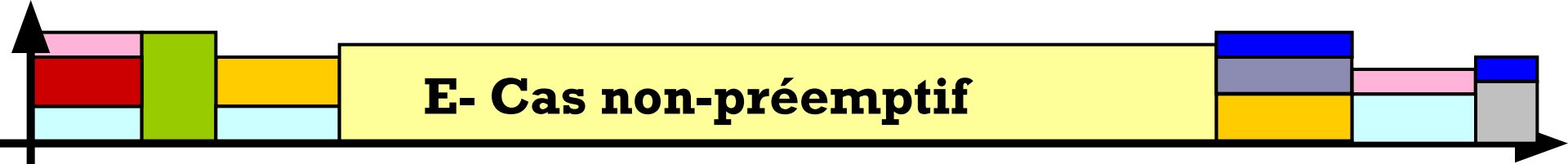
A- Définition du RCPSP

B- Antichaînes

C- Une formulation PLNE pour le cas préemptif

D- Un algorithme BCP pour le cas préemptif

E- Formulation et pistes pour le cas non-préemptif



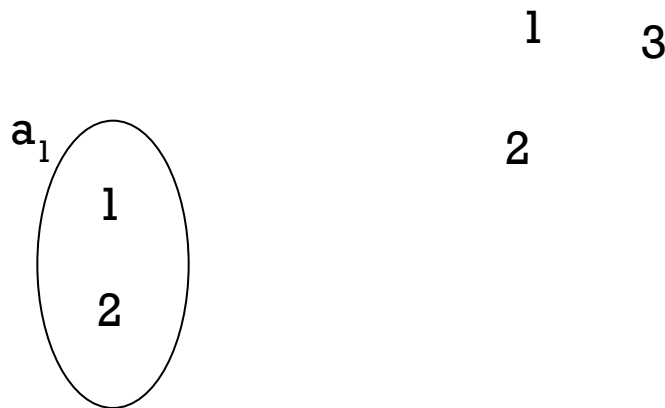
E- Cas non-préemptif

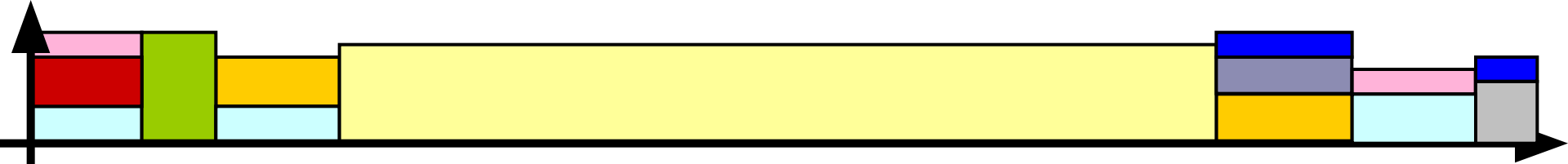
Difficulté du non-préemptif

Dans une séquence ordonnée d'antichaînes,
toutes les antichaînes contenant une même tâche non-préemptive
doivent être consécutives.

Mais ceci crée une sorte de « contrainte de précédence »

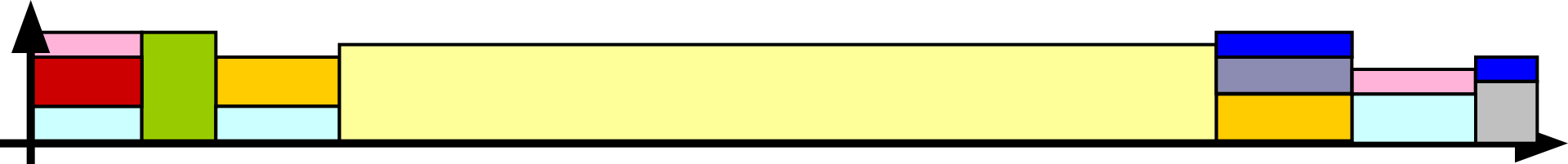
Par exemple pour ce graphe de précédence



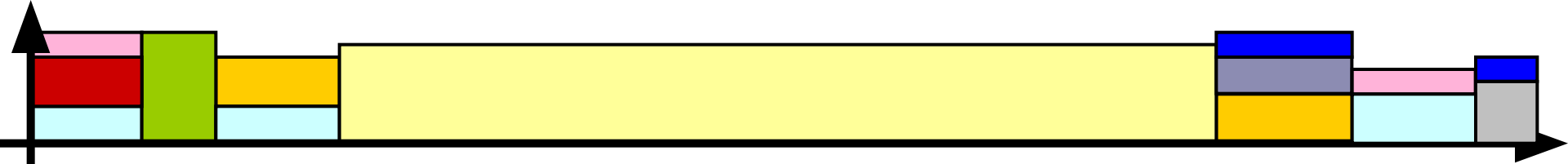


Relation R_v

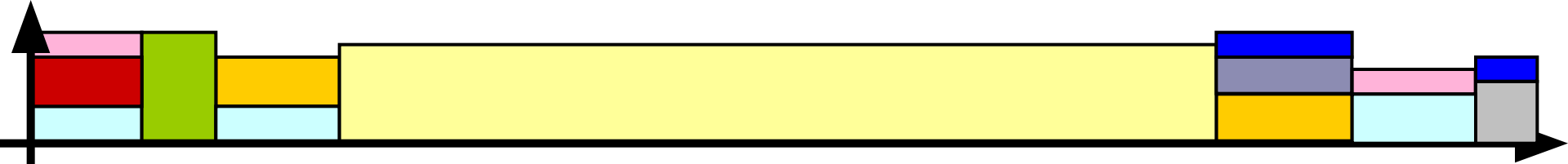
Propag



Ex



Thm



Plne



Conclusion et perspectives

Conclusion

Un algorithme BCP pour le RCPSP préemptif
Permettant de résoudre exactement le problème sans le discrétiser

Perspectives

Tests expérimentaux

Etendre les résultats et l'expérimentation au cas non-préemptif

Ajouter des contraintes de coupes pour éviter le branchement

Mieux résoudre le knapstable